

Max Absolute Difference of Two Non-Overlapping Subarrays

Problem

Given an integer array `arr`, find two **non-overlapping contiguous subarrays** such that the absolute difference between their sums is maximum.

We want:

```
abs(sum(subarray1) - sum(subarray2))
```

Example

```
arr = [-2, -3, 4, -1, -2, 1, 5, -3]
```

Choose:

```
Subarray 1 = [-2, -3]      → sum = -5  
Subarray 2 = [4, -1, -2, 1, 5] → sum = 7
```

Therefore:

```
abs(-5 - 7) = 12
```

Answer:

```
12
```

1. Pattern Name

Main Pattern

Split + Prefix/Suffix Kadane DP

Related patterns:

- Kadane's Algorithm
- Maximum/Minimum Subarray Sum
- Prefix/Suffix DP
- Non-overlapping Subarray Optimization
- Extreme Value Reduction

2. Brute Force Thinking

A subarray is represented by:

```
start index → end index
```

For two subarrays:

```
First subarray → l1, r1  
Second subarray → l2, r2
```

We want:

```
sum1 = sum(arr[l1 ... r1])  
sum2 = sum(arr[l2 ... r2])
```

and maximize:

```
abs(sum1 - sum2)
```

Since the subarrays must be non-overlapping, we can arrange them as:

```
First Subarray | Second Subarray  
l1 ..... r1    l2 ..... r2
```

with:

```
l2 > r1
```

This guarantees that the two subarrays do not overlap.

3. Brute Force Code

```
def brute_force(arr):  
    n = len(arr)  
    answer = 0  
  
    # Choose first subarray  
    for l1 in range(n):  
        for r1 in range(l1, n):  
  
            # Choose second subarray  
            # It must start after r1  
            for l2 in range(r1 + 1, n):  
                for r2 in range(l2, n):  
  
                    sum1 = sum(arr[l1:r1 + 1])  
                    sum2 = sum(arr[l2:r2 + 1])  
  
                    difference = abs(sum1 - sum2)  
  
                    answer = max(answer, difference)  
  
    return answer
```

Complexity

Time Complexity: $O(n^4)$
Space Complexity: $O(1)$

4. First Optimization: Prefix Sum

The repeated:

```
sum(arr[l:r + 1])
```

takes $O(n)$ time.

We can calculate any subarray sum in $O(1)$ using prefix sums.

For:

```
arr = [2, -1, 3, 4]
```

build:

```
prefix = [0, 2, 1, 4, 8]
```

Definition:

```
prefix[i] = sum of elements before index i
```

Therefore:

```
sum(l, r) = prefix[r + 1] - prefix[l]
```

Example:

```
sum(1, 3)
```

The subarray is:

```
[-1, 3, 4]
```

Calculation:

```
prefix[4] - prefix[1]  
= 8 - 2  
= 6
```

This improves:

```
Subarray sum calculation:  $O(n) \rightarrow O(1)$ 
```

But we still have too many possible pairs of subarrays.

5. Main Mathematical Observation

Suppose the first subarray has sum:

A

and the second subarray has sum:

B

We want:

$\text{abs}(A - B)$

Suppose:

$A = 10$

Possible values of B:

$B = [-5, 2, 7, 20]$

Calculate:

```
abs(10 - (-5)) = 15
abs(10 - 2)    = 8
abs(10 - 7)    = 3
abs(10 - 20)   = 10
```

The maximum difference comes from an extreme value of B.

Therefore:

B should be either:

1. Minimum possible B
2. Maximum possible B

Mathematically:

```
max(abs(A - B))  
=  
max(  
    A - min(B),  
    max(B) - A  
)
```

So instead of comparing every possible subarray sum with every other subarray sum, we only need:

```
Minimum subarray sum  
Maximum subarray sum
```

6. The Split Observation

The two subarrays must be non-overlapping.

Therefore, there is always a boundary:

```
LEFT | RIGHT
```

For example:

```
[-2, -3, 4] | [-1, -2, 1, 5, -3]
```

Any valid pair of non-overlapping subarrays can be viewed around some split.

The first subarray lies completely in the left portion.

The second subarray lies completely in the right portion.

For every split, we need:

```
leftMax  
leftMin  
  
rightMax  
rightMin
```

Where:

```
leftMax = maximum sum of any subarray in the left portion
leftMin = minimum sum of any subarray in the left portion

rightMax = maximum sum of any subarray in the right portion
rightMin = minimum sum of any subarray in the right portion
```

7. Why Do We Need Minimum and Maximum From Both Sides?

Suppose:

```
Left possible subarray sums:
[-5, -2, 4]
```

and:

```
Right possible subarray sums:
[-3, 1, 7]
```

We want:

```
abs(leftSum - rightSum)
```

The largest difference can happen in two directions.

Case 1: Left Has the Larger Sum

Choose:

```
leftMax
rightMin
```

```
leftMax - rightMin
= 4 - (-3)
= 7
```

Case 2: Right Has the Larger Sum

Choose:

```
rightMax  
leftMin
```

```
rightMax - leftMin  
= 7 - (-5)  
= 12
```

Therefore:

```
candidate = max(  
    leftMax - rightMin,  
    rightMax - leftMin  
)
```

This is equivalent to checking:

```
max(abs(leftSum - rightSum))
```

over all possible left and right subarray sums.

8. Final Formula

For every split:

```
LEFT | RIGHT
```

calculate:

```
candidate = max(  
    leftMax - rightMin,  
    rightMax - leftMin  
)
```

Then:


```
answer = max(answer, candidate)
```

The two expressions represent the two opposite extreme combinations:

```
Largest left sum - Smallest right sum  
Largest right sum - Smallest left sum
```

9. Finding leftMax

leftMax[i] means:

Maximum sum of any contiguous subarray completely inside arr[0...i].

Example:

```
arr = [1, -2, 3]
```

All subarrays:

```
[1]          = 1  
[-2]         = -2  
[3]          = 3  
  
[1, -2]      = -1  
[-2, 3]      = 1  
  
[1, -2, 3]   = 2
```

Therefore:

```
leftMax[2] = 3
```

because:

```
[3] = 3
```

is the maximum subarray inside:

```
[1, -2, 3]
```

10. Kadane for leftMax

Normal Kadane:

```
current = max(arr[i], current + arr[i])
```

At every index:

```
Option 1: Start a new subarray at arr[i]  
Option 2: Extend the previous subarray
```

Then:

```
best = max(best, current)
```

stores the best subarray sum found so far.

```
left_max = [0] * n  
  
current = arr[0]  
best = arr[0]  
  
left_max[0] = best  
  
for i in range(1, n):  
    current = max(arr[i], current + arr[i])  
    best = max(best, current)  
  
    left_max[i] = best
```

11. Finding leftMin

We use the same Kadane idea, but find the minimum subarray sum.

Maximum Kadane:

```
current = max(arr[i], current + arr[i])
best = max(best, current)
```

Minimum Kadane:

```
current = min(arr[i], current + arr[i])
best = min(best, current)
```

Code:

```
left_min = [0] * n

current = arr[0]
best = arr[0]

left_min[0] = best

for i in range(1, n):
    current = min(arr[i], current + arr[i])
    best = min(best, current)

    left_min[i] = best
```

Meaning:

```
left_min[i]
```

is:

Minimum sum of any contiguous subarray completely inside `arr[0...i]`.

12. Why Do We Need Right Arrays?

Consider a split:

```
arr[0 ... i] | arr[i + 1 ... n - 1]
```

The left side is:

```
arr[0 ... i]
```

The right side is:

```
arr[i + 1 ... n - 1]
```

We already calculate:

```
leftMax[i]  
leftMin[i]
```

for the left side.

Now we need:

```
rightMax[i + 1]  
rightMin[i + 1]
```

for the right side.

Therefore, we calculate the right arrays from:

```
right → left
```

13. Finding `rightMax`

`rightMax[i]` means:

Maximum sum of any contiguous subarray completely inside `arr[i...n-1]`.

Since we need suffix information, we iterate backward:

```
for i in range(n - 2, -1, -1):
```

Code:

```
right_max = [0] * n

current = arr[n - 1]
best = arr[n - 1]

right_max[n - 1] = best

for i in range(n - 2, -1, -1):
    current = max(arr[i], current + arr[i])
    best = max(best, current)

    right_max[i] = best
```

14. Finding rightMin

Similarly:

```
rightMin[i]
```

means:

Minimum sum of any contiguous subarray completely inside `arr[i...n-1]`.

Use minimum Kadane while moving backward:

```
right_min = [0] * n

current = arr[n - 1]
best = arr[n - 1]

right_min[n - 1] = best

for i in range(n - 2, -1, -1):
    current = min(arr[i], current + arr[i])
    best = min(best, current)

    right_min[i] = best
```

15. Dry Run

Consider:

```
arr = [1, -2, 3, -1, 2]
```

Prefix Maximum

```
leftMax = [1, 1, 3, 3, 4]
```

Meaning:

```
leftMax[0] = 1
```

Maximum subarray in:

```
[1]
```

is:

```
1
```

```
leftMax[1] = 1
```

Maximum subarray in:

```
[1, -2]
```

is:

```
[1] = 1
```

```
leftMax[2] = 3
```

Maximum subarray in:

```
[1, -2, 3]
```

is:

```
[3] = 3
```

```
leftMax[3] = 3  
leftMax[4] = 4
```

Prefix Minimum

```
leftMin = [1, -2, -2, -2, -2]
```

For example:

```
leftMin[2] = -2
```

because the minimum subarray sum in:

```
[1, -2, 3]
```

is:

```
[-2] = -2
```

Suffix Maximum

```
rightMax = [4, 4, 4, 2, 2]
```

For example:

```
rightMax[2] = 4
```

because:

```
[3, -1, 2] = 4
```

Suffix Minimum

```
rightMin = [-2, -2, -1, -1, 2]
```

For example:

```
rightMin[2] = -1
```

because:

```
[-1] = -1
```

is the minimum subarray sum inside:

```
[3, -1, 2]
```

16. Evaluate Every Split

The split is between:

```
i and i + 1
```

Split 1

```
[1] | [-2, 3, -1, 2]
```


Use:

```
left_max[0]
left_min[0]

right_max[1]
right_min[1]
```

Candidate:

```
max(
    left_max[0] - right_min[1],
    right_max[1] - left_min[0]
)
```

Split 2

```
[1, -2] | [3, -1, 2]
```

Use:

```
left_max[1]
left_min[1]

right_max[2]
right_min[2]
```

Candidate:

```
max(
    left_max[1] - right_min[2],
    right_max[2] - left_min[1]
)
```

Split 3

```
[1, -2, 3] | [-1, 2]
```

Use:

```
left_max[2]
left_min[2]

right_max[3]
right_min[3]
```

Candidate:

```
max(
    left_max[2] - right_min[3],
    right_max[3] - left_min[2]
)
```

Split 4

```
[1, -2, 3, -1] | [2]
```

Use:

```
left_max[3]
left_min[3]

right_max[4]
right_min[4]
```

Candidate:

```
max(
    left_max[3] - right_min[4],
    right_max[4] - left_min[3]
)
```

17. Final Optimized Python Code

```
def max_absolute_diff(arr):
    n = len(arr)
```

```
left_max = [0] * n
left_min = [0] * n
right_max = [0] * n
right_min = [0] * n

# Maximum subarray sum for every prefix
current = arr[0]
best = arr[0]

left_max[0] = best

for i in range(1, n):
    current = max(arr[i], current + arr[i])
    best = max(best, current)

    left_max[i] = best

# Minimum subarray sum for every prefix
current = arr[0]
best = arr[0]

left_min[0] = best

for i in range(1, n):
    current = min(arr[i], current + arr[i])
    best = min(best, current)

    left_min[i] = best

# Maximum subarray sum for every suffix
current = arr[n - 1]
best = arr[n - 1]

right_max[n - 1] = best

for i in range(n - 2, -1, -1):
    current = max(arr[i], current + arr[i])
    best = max(best, current)

    right_max[i] = best

# Minimum subarray sum for every suffix
current = arr[n - 1]
best = arr[n - 1]

right_min[n - 1] = best

for i in range(n - 2, -1, -1):
    current = min(arr[i], current + arr[i])
    best = min(best, current)

    right_min[i] = best
```

```
# Try every possible split
answer = 0

for i in range(n - 1):

    candidate = max(
        left_max[i] - right_min[i + 1],
        right_max[i + 1] - left_min[i]
    )

    answer = max(answer, candidate)

return answer
```

18. Code-Building Logic

The code was built in this order:

1. Need maximum subarray sum on every left side
↓
left_max
2. Need minimum subarray sum on every left side
↓
left_min
3. Need maximum subarray sum on every right side
↓
right_max
4. Need minimum subarray sum on every right side
↓
right_min
5. Try every boundary
↓
calculate candidate
↓
update answer

The final formula is:

```
candidate = max(
    left_max[i] - right_min[i + 1],
    right_max[i + 1] - left_min[i]
)
```

19. Why Are the Loop Directions Different?

Left Arrays

We need information about:

```
arr[0 ... i]
```

This is prefix information.

Therefore:

```
left → right
```

```
for i in range(1, n):
```

Right Arrays

We need information about:

```
arr[i ... n - 1]
```

This is suffix information.

Therefore:

```
right → left
```

```
for i in range(n - 2, -1, -1):
```

20. Split Index Visualization

Suppose:

```
arr = [a, b, c, d, e]
```

Split after index 1:

```
[a, b] | [c, d, e]
  0  1   2  3  4
```

The split index is:

```
i = 1
```

Left information:

```
left_max[1]
left_min[1]
```

Right information:

```
right_max[2]
right_min[2]
```

Therefore:

```
candidate = max(
    left_max[1] - right_min[2],
    right_max[2] - left_min[1]
)
```

General form:

```
left side  = [0 ... i]
right side = [i + 1 ... n - 1]
```

21. Complexity

We perform four linear passes:

1. left_max $\rightarrow O(n)$
2. left_min $\rightarrow O(n)$
3. right_max $\rightarrow O(n)$
4. right_min $\rightarrow O(n)$
5. split scan $\rightarrow O(n)$

Therefore:

Time Complexity: $O(n)$
Space Complexity: $O(n)$

22. Pattern Recognition

When you see:

Choose two non-overlapping subarrays

immediately think:

There must be a boundary between them.

Visualize:

LEFT | RIGHT

Then ask:

What information do I need from the left side?
What information do I need from the right side?

If the objective involves:

maximum difference
absolute difference
maximum sum difference

ask:

Do I only need extreme values?

For this problem:

```
leftMax  
leftMin  
rightMax  
rightMin
```

Then ask:

Can Kadane calculate those extremes?

Yes.

Therefore:

```
Prefix Maximum Kadane  
Prefix Minimum Kadane  
  
Suffix Maximum Kadane  
Suffix Minimum Kadane
```

23. Quick Identification Template

Trigger Words

Look for:

```
two subarrays  
non-overlapping  
maximum difference  
absolute difference  
maximum sum difference
```

Recognition Process

1. Non-overlapping?
↓
2. Create a split:
LEFT | RIGHT
↓
3. Need extreme values from both sides?
↓
4. Maximum subarray sum?
↓
5. Use Kadane
↓
6. Minimum subarray sum?
↓
7. Use Minimum Kadane
↓
8. Need every split?
↓
9. Build prefix/suffix DP arrays

24. General Template

```
def solve(arr):
    n = len(arr)

    left_max = [0] * n
    left_min = [0] * n

    right_max = [0] * n
    right_min = [0] * n

    # Prefix maximum Kadane
    current = arr[0]
    best = arr[0]

    left_max[0] = best

    for i in range(1, n):
        current = max(arr[i], current + arr[i])
        best = max(best, current)
        left_max[i] = best

    # Prefix minimum Kadane
    current = arr[0]
    best = arr[0]

    left_min[0] = best

    for i in range(1, n):
```

```
        current = min(arr[i], current + arr[i])
        best = min(best, current)
        left_min[i] = best

# Suffix maximum Kadane
current = arr[n - 1]
best = arr[n - 1]

right_max[n - 1] = best

for i in range(n - 2, -1, -1):
    current = max(arr[i], current + arr[i])
    best = max(best, current)
    right_max[i] = best

# Suffix minimum Kadane
current = arr[n - 1]
best = arr[n - 1]

right_min[n - 1] = best

for i in range(n - 2, -1, -1):
    current = min(arr[i], current + arr[i])
    best = min(best, current)
    right_min[i] = best

answer = 0

# Check every split
for i in range(n - 1):
    answer = max(
        answer,
        left_max[i] - right_min[i + 1],
        right_max[i + 1] - left_min[i]
    )

return answer
```

25. Similar Problems

Maximum Subarray Problems

- Maximum Subarray — Kadane's Algorithm
- Maximum Circular Subarray Sum
- Maximum Subarray Sum After One Deletion
- Maximum Product Subarray

Prefix/Suffix Kadane Problems

- Maximum Difference Between Two Subarrays
- Maximum Difference of Two Non-Overlapping Subarrays
- Maximum Sum of Two Non-Overlapping Subarrays
- Maximum Sum of Three Non-Overlapping Subarrays

Prefix/Suffix Extreme Problems

- Trapping Rain Water
- Product of Array Except Self
- Maximum Difference `arr[j] - arr[i]`
- Best Time to Buy and Sell Stock
- Maximum Difference Between Increasing Elements

26. Real-World Interpretation

Imagine:

```
arr[i] = daily profit or loss
```

You want to choose:

```
one continuous business period
```

and:

```
another separate continuous business period
```

such that the difference between their total profits is maximum.

The periods cannot overlap.

The solution is:

```
Split the timeline into two parts.
```

```
For the left part:
```

```
    find maximum possible profit  
    find minimum possible profit
```

```
For the right part:
```

```
    find maximum possible profit  
    find minimum possible profit
```

Compare the two opposite extreme combinations.

27. Interview Quick Revision Sheet

Problem

Find two non-overlapping contiguous subarrays maximizing:

```
abs(sum1 - sum2)
```

Key Observation

Non-overlapping subarrays imply:

```
LEFT | RIGHT
```

For each split, only extreme values matter:

```
leftMax  
leftMin  
rightMax  
rightMin
```

Formula

```
candidate = max(  
    leftMax - rightMin,  
    rightMax - leftMin  
)
```

How to Calculate

```
leftMax  → Kadane left to right  
leftMin  → Minimum Kadane left to right  
  
rightMax → Kadane right to left  
rightMin → Minimum Kadane right to left
```

Complexity

Time: $O(n)$

Space: $O(n)$

Final Mental Model

Remember this chain:

```
Two non-overlapping subarrays
    ↓
There is a split
    ↓
LEFT | RIGHT
    ↓
Absolute difference
    ↓
Only opposite extremes matter
    ↓
leftMax - rightMin
rightMax - leftMin
    ↓
Need maximum/minimum subarray sums
    ↓
Kadane + Minimum Kadane
    ↓
Need every prefix and suffix
    ↓
Prefix/Suffix DP
    ↓
 $O(n)$  solution
```

One-Line Memory Trick

For every split, find the maximum and minimum subarray sum on both sides, then compare the two opposite extreme combinations.